University of Victoria

ECE 449 PROJECT REPORT FOR CUSTOM PROCESSOR IMPLEMENTATION

Submission Date: March 6th, 2024
Course Section: ECE 449 B01
Instructor: Dr. Amirali Baniasadi

Presented By:
Hoang Tuan Kiet Vu
Phillip Liu
Huss Chaudhry

ELECTRICAL AND COMPUTER ENGINEERING

`

Table of Contents

`

`

List of Figures:

`

Abstract:

This Project Report details the development of a pipeline processor in VHDL. The processor is designed to support custom instruction formats that are categorized as Format A, Format B, and Format L. The processor is built on synchronous digital design principles, the processor was simulated on an FPGA. There were multiple tests that were performed to validate the behavior of each instruction type, with heavy focus on pipeline execution, hazard resolution, and the coordination of control signals. The design also features integration with a bootloader, VGA console, and memory mapped I/0. The project was completed without a step-by-step guide which required a strong understanding of the concepts and debugging. In light of all of these challenges, all goals where achieved and each instruction format operated as it was suppose to.

Contributions:

Format A:	Huss, Hoang
Format B:	Phillip, Hoang
Format L:	Hoang
Report:	Hoang, Phillip, Huss
Slides:	Huss
GitHub:	Phillip

`

1.0 Introduction:

1.1 Design Overview

This project involves designing and implementing a custom pipelined processor that is capable of executing a predefined Instruction Set Architecture (ISA) [1]. The processor is based on a Harvard architecture, which utilizes a dual-ported RAM to separate instruction and data memory access for improved performance [1]. It supports three instruction formats:

1. Format A for arithmetic operations.
2. Format B for branching.
3. Format L for load/store instructions.

The design of this project is built around two components which are, Datapath and Controllers [1]. These two key components work together to execute instructions efficiently. An overview of the processor architecture is presented in Figure 1, which highlights the two core components. The Datapath is responsible for executing instructions and the Controller manages Data flow to make sure there is efficient operations.

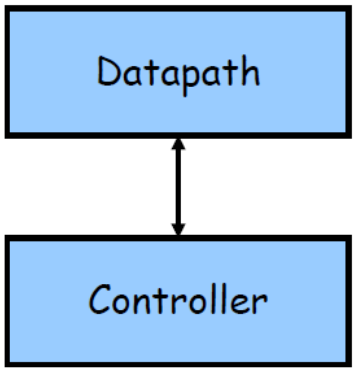


Figure 1: Processor Architecture [1].

1.2 System Architecture:

The system consists of essential components such as, ROM, RAM, an ALU, a Register File, and Memory-Mapped I/O Ports. The ROM contains a basic BIOS that is responsible for loading user programs into the RAM before execution [1]. The RAM is 1024 bytes in size, with a starting at address 0x0400, while the ROM is also 1024 bytes, and it begins at 0x0000 [1]. Memory mapped I/O ports are located at 0xFFF0 and 0xFFF2 [1], which make the communication possible with external devices.

The processor employs a dual-port RAM using Xilinx's XPM_MEMORY_DPDISTRAM macro, where Port A supports both the read and the write operations [1]. This dual-port design prevents instruction fetch delays, ensuring smooth operation. A visualization of the processor's memory system and its connectivity is presented below in Figure 2.

`

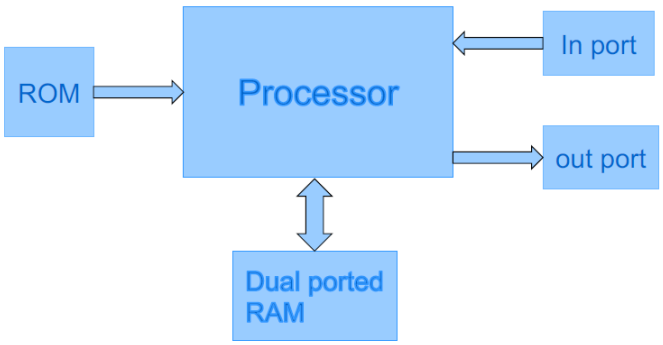


Figure 2: System Description [1].

1.3 Instruction Set and Pipeline Architecture:

As mentioned in design overview (section 1.1), the processor supports three different instruction formats:

1. Format A is the Arithmetic Instructions, its job is to handle computations such ass ADD r3, r2, r1 when values from r2 and r1 are added and stored in r3. This format follows a fixed length encoding [1].
2. Format B is the Branch Instruction, it supports the conditional and unconditional jumps, such as BR r3 + 0x8, that is used to change the control flow [1].
3. Format L is the Load/Store Instruction, it is used for memory access, such as LOAD r1, @r2, where r1 loads data from the memory at the address that is stored in r2 [1].

The processor uses a five stages pipelined execution model (showing in Figure 3) to better enhance the performance. The Five stages are:

1. Instruction Fetch (IF): The next instruction is fetched from ROM.
2. Instruction Decode (ID): The opcode and operands are identified.
3. Execution (EX): The ALU operations or address calculations are performed.
4. Memory Access (MEM): The Data is read from or written to memory
5. Write Back (WB): The results are stored back in the register file.

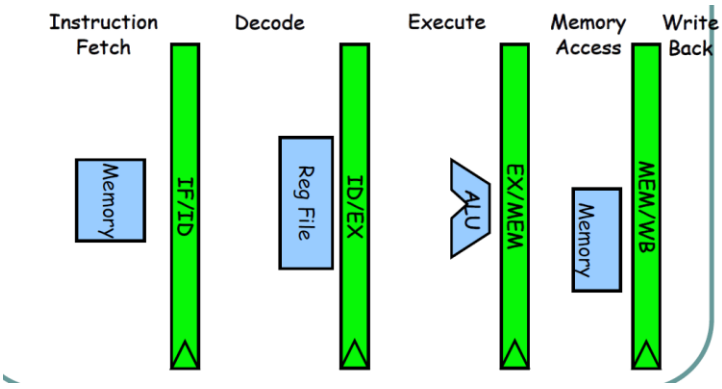


Figure 3: Five Stags Datapath [1].

`

1.4 Control Flow and Hazard Handling:

The processor's Control Unit plays a very important role in the execution of flow by generating the required control signals. To make sure there is smooth instruction execution; the Control Unit utilizes a Finite State Machine (FSM) that dictates the processor's current execution state and manages instruction flow through the pipeline. The Control Unit makes sure there are efficient instruction sequencing and proper data handling by coordinating the various stages of the execution process [1]. The pipeline introduce potential execution hazards; therefore, the design has several strategies to mitigate their effects:

- Data Hazards come up when an instruction requires the result of a previous instruction that has not yet completed execution. If this is left unaddressed, this could lead to incorrect computations or stalled execution. To resolve this, techniques like data forwarding allow dependent instructions to access intermediate results directly, while pipeline stalls temporarily halt execution to prevent errors [1].
- Control Hazards occur due to branch instructions that modify the normal flow of execution. If the pipeline processes an instruction speculatively and the branch outcome differs, unnecessary instructions may be executed; therefore, wasting cycles. To minimize the disruption, the design incorporates branch delay techniques, which either introduce deliberate stalls or restructure instruction scheduling to ensure proper execution flow [1].
- Structural Hazards emerge when hardware resources are insufficient to handle simultaneous instruction execution which causes contention between different operations. However, this design avoids such issues by implementing a Harvard architecture, which provides separate pathways for instruction and data memory access; therefore, eliminating resource conflicts and improving overall efficiency.

Through these hazard-mitigation techniques, the processor maintains a stable and efficient execution pipeline, reducing delays and optimizing performance.

2.0 Objective:

The aim of this project is to design and implement a pipeline RISC processor that efficiently executes a predefined Instruction Set Architecture (ISA). The processor will support arithmetic, branching and memory operations, and use a structures five stage pipeline while effectively managing instructions dependencies and associated hazards.

2.1: Key Objectives:

- Developing an optimized Datapath and control unit to execute instructions efficiently [1].
- Implementing a five-stage pipeline to improve instruction that are put through the pipeline [1].
- Integrating memory mapped I/O and dual ported ram for seamless data and instruction access [1].
- Ensuring robust hazard detection and resolution mechanisms that handle both data and control dependencies [1].
- Testing and verification of the processor's performance through simulations and synthesis to make sure everything is working correctly and efficiently [1].

`

When everything is put together, the group is hopeful that the final outcome of the project will be a working processor implementation that is capable of executing a variety of instructions while maintaining pipeline efficiency and correctness!

3.0 System Overview:

The processor that was developed for this project follows a five-stage pipelined Harvard architecture. This architecture supports three primary instruction formats:

- Format A for arithmetic
- Format B for branching
- Format L for load/store operations

The instruction Fetch, Decode, Execute, Memory Access, and Write Back, the design enables instruction-level parallelism by segmenting execution into distinct phases.

At the core of this architecture is a dual-ported RAM, that serves as the data and instruction storage through the memory-mapped access. A basic BIOS that is stored in the ROM handles the initial loading and the execution of user programs, and communication with external hardware occurs through the memory mapped I/O, which has designated ports at addresses 0xFFF0 (input) and 0xFFF2 (output). The Top-Level Architecture diagram (see Figure 4) visually outlines the processor's data pathways and control signal flow across the pipeline stages.

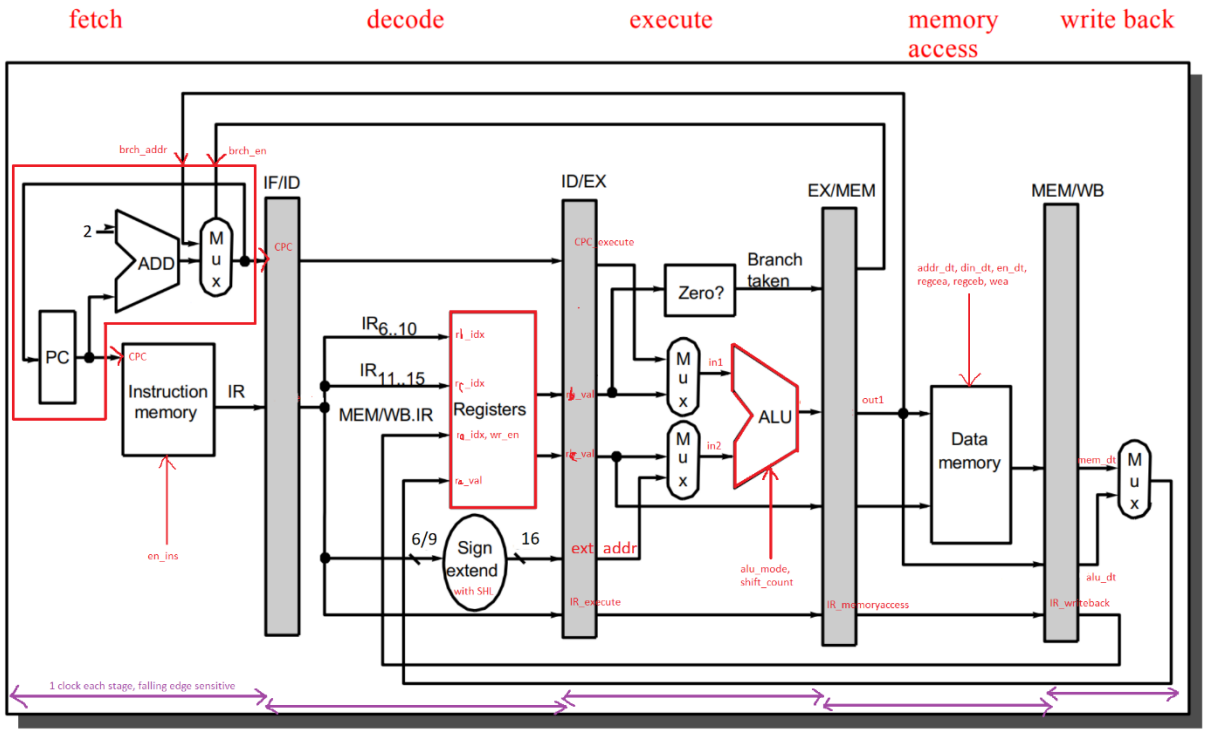


Figure 4: Top-Level Architecture

`

The Datapath serves as the core of the processor architecture; it executes all of the instructions across all of the formats. It integrates all of the essential modules that are responsible for the data manipulation, memory access, register control, and instruction flow. To keep up with the performance and to enable the instruction processing (which work at the same time as each other), each pipeline stage is linked by dedicated their respected pipeline registers.

3.1 Format A:

The main purpose of format a is to serve as the primary instruction format that is used for executing arithmetic and logical operations between registers in the 16-bit processor. Format A uses a three-register structure (see Figure 5); it has two source registers (RB and RC) which supply the operands. It also uses a destination (RA) which receives the results. The typical operations under this format include:

- ADD
- SUB
- MUL
- NAND
- SHL
- SHR
- TEST
- MOV

These operations combined form the foundation for the data processing tasks. Each Format A instruction spans 16 bits and it follows a consistent encoding scheme. The upper seven bits (bits 15 through 9) define the opcode, followed by three 3-bit fields that specify RA, RB, and RC (see Figure 5).

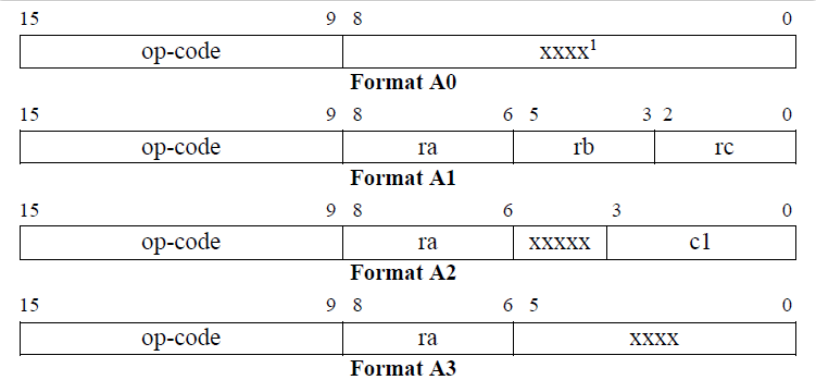


Figure 5: Format A Registers

The decode stage of the code interprets the instruction fields to correctly identify the required source registers and the destination register that is used for storing the result. The execution begins in the Fetch stage, where the PC_file.vhd module generates the address that are used to access the instructions from memory. If there is no branch or stall condition that is active, the program counter advances by two on each clock cycle (see Figure 6).

`

```
if (brch_en = '1') then
    current_PC <= brch_addr;
else
    current_PC <= std_logic_vector(signed(current_PC) + 2);
end if;
```

Figure 6: PC Code

The PC address in figure 6 is used to retrieve the correct instruction from either ROM or RAM, depending on stall conditions that are present, through a multiplexer that is defined in MUX_file.vhd. The selected instruction is then loaded into the IR register within topmodule.vhd, which prepares if for the decode phase. During the decoding stage, the instruction is tasked to extract information form the relevant register indices. The register file is accessed using the code in Figure 7.

```
rb_idx <= IR(5 downto 3);
rc_idx <= IR(2 downto 0);
ra_idx_execute <= '0' & IR(8 downto 6);
```

Figure 7: Code Used to Access Register

This makes sure that the correct source and destination registers are passed forward into the pipeline. The REGISTER_file.vhd module handles reading the values from the register file using the code in Figure 8.

```
case rd_index1 is
    when "000" => rd_data1 <= reg_file(0);
    when "001" => rd_data1 <= reg_file(1);
    ...
end case;
```

Figure 8: Code Which Reads Values of Register

The source operands (rb_val and rc_val) are then sent to the ALU for processing, while the destination register index (ra_idx) is held for the Write-Back stage. In the Execute stage, the ALU receives both input values and it then performs the specified operation based on the instruction's opcode. Within ALU_file.vhd, a case statement driven by the alu_mode signal determines which arithmetic or logic function to execute (See Figure 9).

`

```
case alu_mode is
    when "001" => temp <= signed(in1) + signed(in2); -- ADD
    when "010" => temp <= signed(in1) - signed(in2); -- SUB
    when "011" => temp <= signed(in1) * signed(in2); -- MUL
    when "100" => temp <= not (in1 and in2);         -- NAND
```

Figure 9: ALU operations based on alu_mode control

The result of the ALU operation is stored in out1, then captured by the alu_dt signal and forwarded to the Memory stage. Since Format A instructions do not interact with data memory, this stage is typically bypassed, except for specific cases like the OUT instruction. During the Write Back stage, the destination register (RA) is updated with the ALU result. This operation is controlled by logic within topmodule.vhd, which checks the opcode and enables the write accordingly; it is shown below in Figure 10.

```
when "0000001" => -- ADD
    ra_idx <= ra_idx_writeback(2 downto 0);
    ra_val <= alu_dt;
    wr_en <= '1';
```

Figure 10: ADD result routed to RA register

When the wr_en is asserted, the REGISTER_file.vhd module writes the computed value into the destination register using the wr_index as the address and the wr_data as the input. An important part of the architecture is the built-in support that is used for hazard detection and data forwarding, which is handled within topmodule.vhd. If there is a source operand that is pending from a prior instruction, the processor can either forward the result from the Write Back stage or initiate a stall if the instruction involves a memory load. This mechanism preserves execution correctness without requiring a pipeline flush.

In summary, Format A instructions are very tightly integrated into the processor's five-stage pipeline, which use nearly all major modules, including the program counter, memory interface, register file, ALU, control unit, and output logic. The instruction journey starts with memory fetch, which is followed by decoding, ALU execution, and it concludes with writing the result back to the register file. This format forms the backbone of arithmetic and logical computation within the architecture (see Figure 11).

`

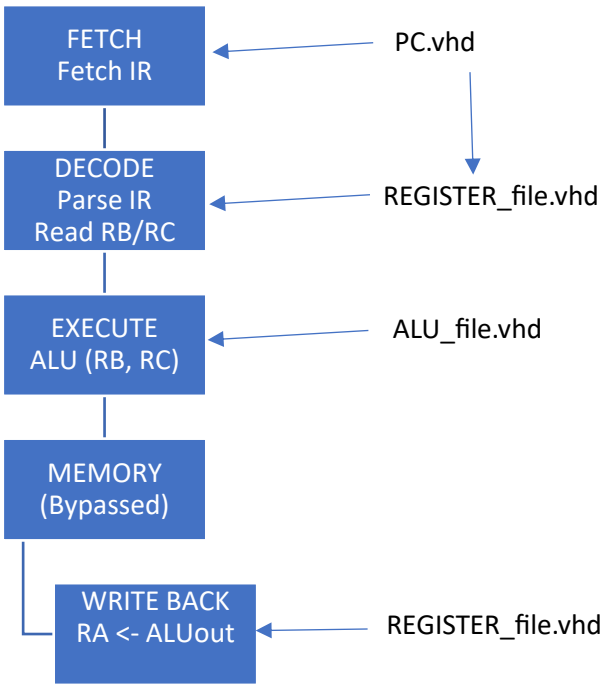


Figure 11: Format A Flowchart

3.2 Format B:

For Format B, the instructions are designed to use a single source and a single destination register, with part of the instruction reserved for an immediate value or memory offset. This format supports key operations such as:

- LOAD: Retrieves a value from memory into a register
- STORE: Writes a register's value to a specified memory address
- LOADIMM: Loads an immediate constant directly into a register
- MOV: Transfers data between two registers

Each instruction uses 16 bits, and it has designated fields for the opcode, register identifiers, and the address segment. Similar to the Format A, Format B operations also start at the fetch state in the pipeline, it is handled in the PC_file and Mux logic. The Program Counter (PC) pics the next instruction address unless there is a stall or a branch is detected, the code is presented in Figure 12.

```
component PC_file port (
    brch_addr: in std_logic_vector(15 downto 0);
    brch_en: in std_logic;
    rst: in std_logic;
    clk: in std_logic;
    stall: in std_logic;
    CPC: out std_logic_vector(15 downto 0)
);
```

Figure 12: Program Counter (PC) component interface

`

The instruction is fetched either from RAM or ROM through the MUX:

```
MUX_ROMRAM: MUX_file port map(IR_ROM, IR_RAM, stall_MUX, CPC(10), IR);
```

Figure 13: MUX selects ROM or RAM instruction source

Depending on the opcode (IR(15 downto 9)), in the decode stage, Format B instructions are identified and the registers are set accordingly.

```
when "0010000" => -- LOAD
    rb_idx <= IR(5 downto 3);
    rb_idx_execute <= '0' & IR(5 downto 3);
    ra_idx_execute <= '0' & IR(8 downto 6);
```

Figure 14: LOAD instruction sets RB and RA indices

Figure 14 shows the destination (ra) and source address register (rb) are extracted from the instruction and then passed onto the next stage. The ALU_file is continued and still used in Format B to compute the effective addresses and manage the data flow logic. For example, during LOAD and STORE, the ALU computes the effective memory address using the value in the base register (see Figure 15).

```
alu_mode <= "111";  -- ALU mode set to pass-through or logical
hold_flag <= '1';
in1 <= rb_val; -- base address
```

Figure 15: ALU setup for memory address pass-through

The ALU combines parts of the instruction as per the IR(8) flag, determining if the upper or lower byte is being set for the LOADIMM:

```
if (IR_execute(8) = '1') then
    in1 <= X"00" & rb_val(7 downto 0);
    in2 <= IR_execute(7 downto 0) & X"00";
```

Figure 16: Immediate value formatting for memory addressing

Format B uses both ports of RAM, utilizing RAM_file, for memory operations. While in the LOAD, the calculated address is sent to addr_dt, and then the retrieved memory value is then loaded back into the destination register in the write-backstage:

`

```
when "0010000" => -- LOAD
    addr_dt <= out1;
    alu_dt <= out1;
```

Figure 17: Output assigned to address and ALU data

Then in the write-back phase:

```
when "0010000" => -- LOAD
    if (alu_dt = X"FFF0") then
        ra_val <= X"000" & num;
    else
        ra_val <= mem_dt;
    end if;
    ra_idx <= ra_idx_writeback(2 downto 0);
    wr_en <= '1';
```

Figure 18: Conditional memory load and register write

The STORE operation transfers data from rc_val to din_dt and it is also handled here:

```
when "0010001" => -- STORE
    addr_dt <= out1;
    din_dt <= out2;
    wr_mem_en <= "1";
```

Figure 19: STORE instruction assigns data and address for memory write

For MOV and LOADIMM, the data manipulation occurs in the ALU and the execute stage, with the results being stored in ra_val and written back via the register file. Format B also supports the direct interaction with the external data sources like the memory and the immediate inputs. This bridges the computational logic of Format A with the external environment. It also provides the flexibility in the writing of the programs that require memory access, immediate constants, or the data movement between registers. By isolating the data operations from the core arithmetic logic, Format B instructions allow the processor pipeline to handle the memory access and the data initialization without any interference with ALU-intensive tasks. This separation simplifies instruction decoding and keeps the architecture modular and scalable.

`

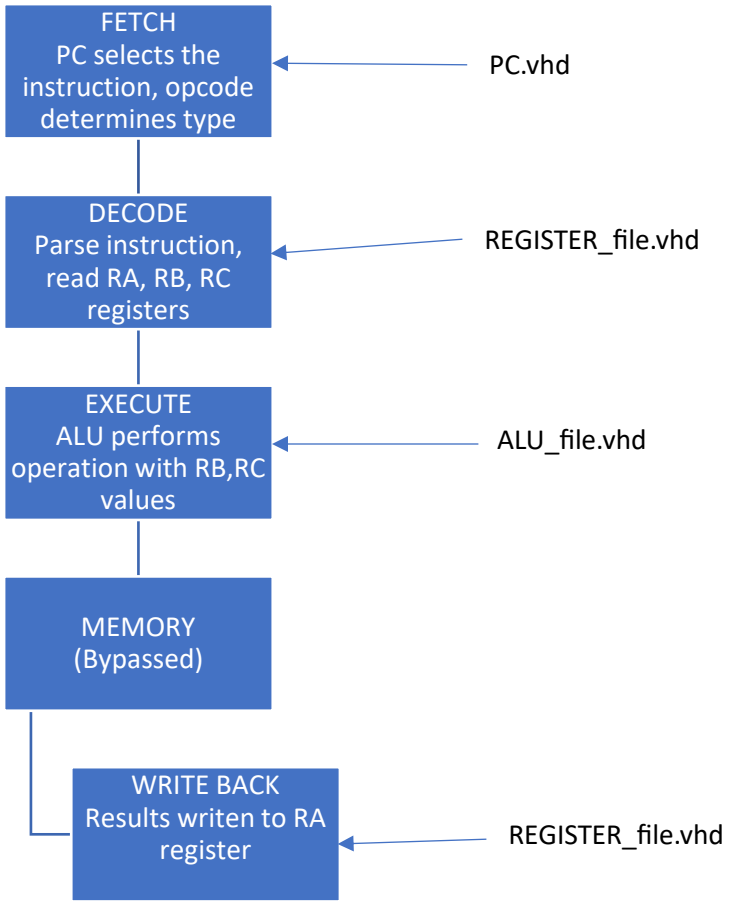


Figure 20: Format B Flowchart

The Format B instructions are structures in a manner so that it operates with two source registers and a destination register. It is primarily used for ALU-baser operations like ADD, SUB, MUL, NAND, SHL, and SHR. Each of the instruction includes three registers fields:

1. RA -> Destination
2. RB -> Source 1
3. RC -> Source 2

Along with the three registers, Format B also uses a 3-bit opcode and a shift count field in the lower bits for shift operations. Just like Format A, these instructions also move through the 5 stages of pipeline (see Figure 20):

- Fetch: The instruction is read from memory using the current program counter (PC). It is then incremented by 2 to point to the next instruction.
- Decode: Register indices are extracted from the instruction; bits [8:6] for RA, [5:3] for RB, and [2,0] for RC. These are used to read values from the register file (REGISTER_file.vhd), assigning them to rb_val and rc_val.
- Execute: The ALU receives the operands and executes the operation specified by the alu_mode signal. If a data dependency is detected, forwarding or stalling is applied to prevent hazards.

`

- Memory Access: This stage is skipped, because Format B instructions do not access data memory.
- Write back: The alu results in written to the destination register (RA), and the overflow flag is updated based on the outcome.

The structure in Figure 20 allows Format B instructions to execute efficiently, with support for hazard detection and forwarding, which makes sure there is very little impact on the performance of Format B.

3.3 Format L:

The Format L instructions are used to manage the immediate data operations and the system-levels of the data movement. These include instructions such as:

- LOAD
- STORE
- LOADIMM
- MOV

These instructions are very important for reading data, writing data to/from the memory, initializing registers with constants, and moving values between registers. Unlike Format A and Format B, Format L integrates the memory access and I/O directly. This frequently involves the external interaction with the system ports or the memory-mapped locations. These instructions are very critical for computational logic and control logic, because they establish how data enters and then exits the processor's internal pipeline.

The instruction for the Format L structure is defined in the top-level controller module (topmodule.vhd). All of the Format L instruction follows a pipeline flow through the same five standard stages of Format A and Format B:

- Fetch
- Decode
- Execute
- Memory access
- Write back

However, what sets format L apart from the Format A and Format B is the way that Format L makes use of the memory and I/O access paths, along with the conditional value selection (e.g. keyboard inputs, numeric ports), which are not typical in pure arithmetic. In addition, Format L instructions introduces and manages data hazards more often. This is especially required in LOAD and STORE because this requires careful forwarding/stalling mechanisms that are handled in the control logic.

Similar to Format A and Format B, Format L also begins at the fetch stage. This is where the program counter (PC.vhd) provides the address for the instruction memory access. The instruction is fetched form either the ROM_file.vhd or RAM_file.vhd (this is dependent of the state of the stall_MUX.vhd) and then stored in the IR register. From this point, it moves onto the Decode Stage, its here the opcode and operand fields are parsed. The fetch stage code can be vied in Figure 21.

`

```
-- FETCH STAGE
PC_module : PC_file port map(brch_addr, brch_en, rst, clk, stall, CPC);

-- ROM and RAM fetch
ROM_module: ROM_file port map(CPC, rst, clk, IR_ROM);
RAM_module: RAM_file port map(addr_dt, CPC, din_dt, wr_mem_en, rst, clk, mem_dt, IR_RAM);

-- Select between ROM and RAM based on stall_MUX
MUX_ROMRAM: MUX_file port map(IR_ROM, IR_RAM, stall_MUX, CPC(10), IR);
```

Figure 21: Instruction fetch using PC, ROM/RAM modules, and MUX control

The decoding of Format L is handled in the process block which is located within the topmodule.vhd. It has the detailed logic for each instruction type. For example, in load instruction (‘0010000”), bits [8:6] show the destination register (ra_idx_execute). The bits [5:3] point to a register that contain the target memory address (rb_idx_execute). In STORE, the memory address is identified by rb_idx_execute, and the value that is going to be stored comes from the register indicated by rc_idx_execute. The LOADIMM instruction then directly embeds the immediate value withing the instruction; therefore, combining register and constant fields for reconstructions. See Figure 22 for the decoding stage code.

```
-- DECODE STAGE
IR_execute <= IR;
CPC_decode <= CPC;

case IR(15 downto 9) is
    when "0010000" => -- LOAD
        rb_idx <= IR(5 downto 3);  -- memory address source
        rb_idx_execute <= '0' & IR(5 downto 3);
        ra_idx_execute <= '0' & IR(8 downto 6);  -- destination register
    when "0010001" => -- STORE
        rb_idx <= IR(8 downto 6);  -- address register
        rb_idx_execute <= '0' & IR(8 downto 6);
        rc_idx <= IR(5 downto 3);  -- source data
        rc_idx_execute <= '0' & IR(5 downto 3);
    when "0010010" => -- LOADIMM
        rb_idx <= "111";
        rb_idx_execute <= "0111";
        ra_idx_execute <= "0111";  -- hardcoded R7 as staging for partial immediate
    when "0010011" => -- MOV
        rb_idx <= IR(5 downto 3);  -- source
        rb_idx_execute <= '0' & IR(5 downto 3);
        ra_idx_execute <= '0' & IR(8 downto 6);  -- destination
    when others =>
        NULL;
end case;
```

Figure 22: Instruction decode logic for LOAD, STORE, LOADIMM, and MOV operations

`

During the execute state, the Format L instructions primarily sets up the address/values for the next phase. The ALU operates in mode 111 which is a passthrough configuration for the LOAD And STORE operations. In this mode, rv_value is used directly as the memory address. This behavior is implemented in the control logic within the topmodule.vhd which is showing below in Figure 23.

```
alu_mode <= "111";
hold_flag <= '1';
if (rb_idx_execute = ra_idx_memoryaccess) then
    -- Handle potential hazard
```

Figure 23: Control logic for hazard detection and ALU mode setup

Two of the most important stages of Format L are the hazard detection and resolution. While the ALU may not fully perform arithmetic in every Format L instruction, it is still a key part of the data path. In the case of the LOADIMM, a special handling makes sure the immediate value is correctly written to the target register. Bit 8 of the instruction determines whether the high or low byte is being set, and the final 16-bit value is then reconstructed by combining parts of the instruction and register using the code in Figure 24.

```
if (IR_execute(8) = '1') then
    in1 <= X"00" & out1(7 downto 0);
    in2 <= IR_execute(7 downto 0) & X"00";
else
    in1 <= out1(15 downto 8) & X"00";
    in2 <= X"00" & IR_execute(7 downto 0);
end if;
```

Figure 24: Immediate vs register operand parsing logic

This logic, in Figure 24, dynamically determines whether the high or low byte is being modified and if it aligns the immediate properly. Moving on to the Memory Access state, which highlights the very distinct role of the Format L instructions. In the LOAD operation, the previously computed address is used to read the data from memory via the RAM_file.vhd. The retrieved value (mem_dt) is them prepared for writing in the cycle which follows. For STORE instructions, the data in out2 is written to memory. If the target address corresponds to a special memory mapped I/O locations, it can then trigger an output action or alter the system state; see figure 25 for the associated code.

`

```
if (out1 = X"FFF2") then
    CPU_output <= out2;
    if (out2 = X"0000") then
        z <= '1';
    else
        z <= '0';
    end if;
    n <= out2(15);
else
    addr_dt <= out1;    -- destination memory address
    din_dt <= out2; -- data to be written
    wr_mem_en <= "1";
end if;
```

Figure 25: Output or memory write logic based on out1

The snippet of code above shows how STORE writes to memory or triggers a zero/negative flag update for system out monitoring. The IN instruction retrieves data from the input_port and then stores it in the designated register, converting the 10-bit input into a 16-bit value by appending the six leading zeroes (“000000”). The last part of Format L is the Write Back Stage; In this stage the Format L instructions update the target register. Except in the case of STORE because it does not involve write back. However, for Load, if the access address is 0xFFF0, the processor uses the num value received from the external switches. This code that implements this presented in Figure 26.

```
if (alu_dt = X"FFF0") then
    ra_val <= X"000" & num;
else
    ra_val <= mem_dt;
end if;
```

Figure 26: Conditional register assignment from memory or constant

The dual path logic allows the processor to source the data from the static memory or user input, depending on the instruction. For MOV operations, which is responsible for transferring values between registers without accessing memory, the ALU forwards the source value to be written into the destination register. The successful write back operation are indicated by setting we_en<=’1’, this allows the REGISTER_file to update the specified register.

The execution of Format L instructions heavily relies on a set of control signals to mange their different behaviours. Alu_mode signal is at the center of this process, which is set to “111” for the passthrough operations like MOV, and to “001” for arithmetic operations such as LOADIMM, where summing high

`

and low byte is required. Its important to note that during these operations, the hold_flag is assessed to supress any unnecessary activity and preserve data integrity, specially in the multi-cycle instructions where the temporary values are staged.

For the memory instructions, signals like s3_mc_rd_sig and s3_mr_wr_sig are used to initiate the read and write requests. The stall_MUX signal plays the most important role in handling instructions involving immediate data or control flow changes for LOADIMM or BR.SUB. It selects between ROM and RAM as the instruction source, making sure it is fetching the correct information. When working together, these control signals all the fine detailed control which is necessary for the specialized execution flow of format L (see Figure 27).

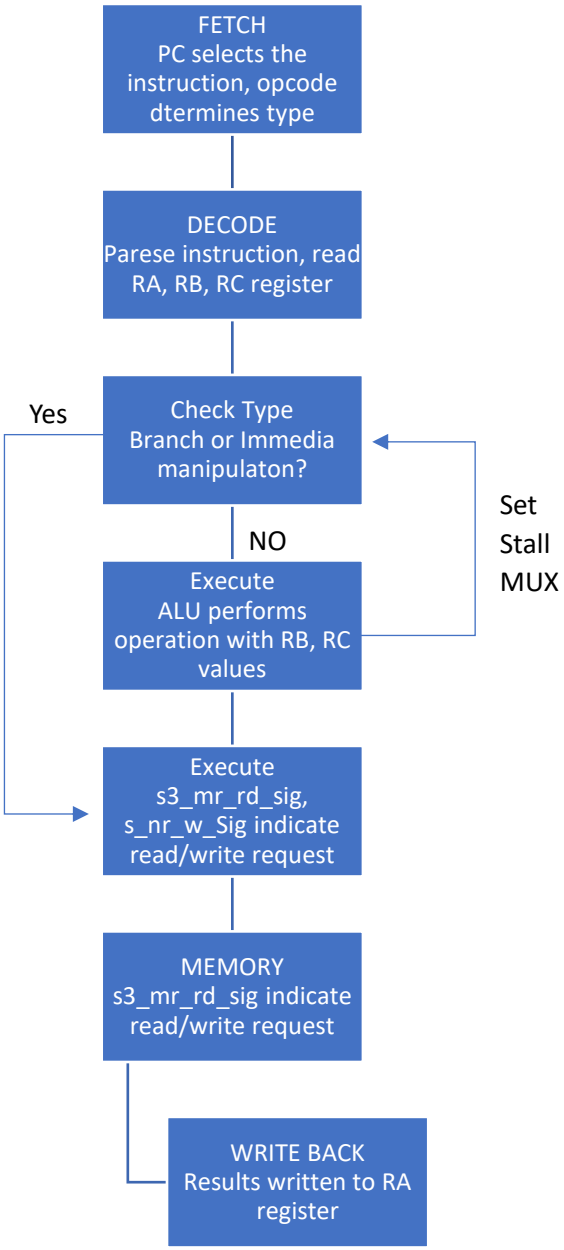


Figure 27: Format L Flowchart

`

Format L plays a very important role in integrating memory and I/) operations, it acts as an essential link between the processor computation and external system. It interacts/engages with all aspects of control logic, including hazard detection, ALU passthrough, memory access cycles, and the special behaviours which are tied to port based I/O. Due the complexity of Format L, it requires close coordination among the ALU, control unit and memory interface.

In conclusion, the implementation of instruction Formats A, B, and L within the pipeline processor highlights a well structured and flexible architecture that is capable of executing a wide range of operations, from arithmetic and memory access to extremely complex control flow. Starting with Format A, it is used for register based arithmetic operations, such as ADD, SUB, and MUL. It also integrates the register file and ALU to handle computations efficiently. These instructions pass through all fife of the pipeline stages with the ALU performing calculations in the Execute stage and the result written back to the proper destination register. The Format A flowchart shows this streamlined data path; it emphasizes the simplicity and the speed of the register-to-register operations in the pipeline. Next, Format B dives into the processor's functionality to include memory operations such as LOAD, STORE, and MOV. These instructions rely on the RAM module and signals like s3_mr_rd_sig and s3_mr_wr_sig to mange the memory reads and writes. Unlike Format A, Format B instructions can bypass the ALU or use it for address computations. The flowchart for Format B shows the complexity of routing data between registers and memory, making sure there is accurate memory access and register updates. When it comes to instructions like LOAD and MOV, the Write Back stage makes sure the DATA is fetched from the memory or transferred between registers is correctly picked. Lastly, Format L is the most complex format. Format L monitors branching and control-flow operations like BR, BR.Z, BR.N BRR, RETURN, and LOADIMM. All of these instructions manipulate the program counter (PC) and rely on the condition flags like, z_flag, n_flag, and o_flag to control the flow. The Format L flowchart shows the conditional paths these instructions take. It also highlights the role of the sign extender (SIGNEXT_File.vhd) for address resolutions and the instruction multiplexer (MUX_file.vhd) for selecting between ROM and RAM. The key control signals like stall_MUX, hold_flag, and brch_en manage pipeline integrity during branching, making sure there is accurate execution.

A timing diagram was also generated to help visualize how these instructions overlap in the pipeline across clock cycles. It highlights the instruction-level parallelism which is achieved through pipelining while managing data hazards and control dependencies.

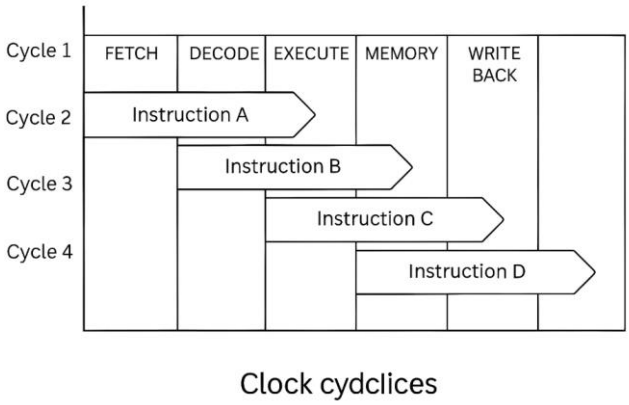


Figure 28: Timing Diagram

`

Together, the detailed flowcharts and VHDL implementations provide a comprehensive view of how our processor architecture handles diverse instruction formats. The modular and synchronized pipeline stages ensure that each instruction type is executed efficiently. By mapping control signals, hardware modules, and data paths for each format, the processor achieves robust performance and flexibility. These visual aids and VHDL references not only validate our implementation but also lay the groundwork for future expansions in instruction sets or architectural optimizations.

4.0 Results:

Before performing the final test for the program, each section of the CPU was tested by using the Simulation function of Vivaldo. Before simulating and of the format, the assembly code given in the lab's webpage has to be reformatted into VDHL with all of the declared gate connected accordingly. Below is a sample test plan provided from the lab.

```
ORG  0x0210
.CODE
        IN R0 ; 02  ; This example tests how data dependencies are handled
        IN R1 ; 03  ; The values to be loaded into the corresponding resgister.
        IN R2 ; 01
        IN R3 ; 05  ;  End of initialization
        ADD R1, R1, R2
        SUB R2, R1, R0
        SUB R1, R3, R2

END
```

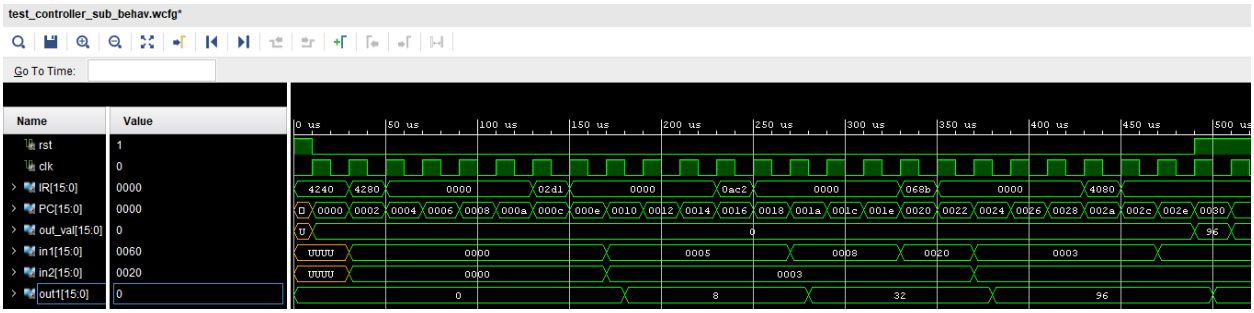
`

4.1 Format A

By using the testbench below, the following simulation results were retrieved

```
library ieee; -- Format A Test bench
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use work.all;
entity test_controller_sub is end test_controller_sub;
architecture behavioural of test_controller_sub is
    component CONTROLLER_file port(
        --input signals
        rst: in std_logic;
        clk: in std_logic;
        IR: in std_logic_vector(15 downto 0);
        --output signal
        PC: out std_logic_vector(15 downto 0);
        out_val: out std_logic_vector(15 downto 0)
    );
    end component;
    signal rst, clk : std_logic;
    signal IR, PC, out_val : std_logic_vector(15 downto 0);
    begin
    u0 : CONTROLLER_file port map(rst, clk, IR, PC, out_val);
    process begin
        clk <= '0'; wait for 10 us;
        clk <= '1'; wait for 10 us;
    end process;
    process begin
        rst <= '1'; IR <= X"4240"; wait until (rising_edge(clk));
        rst <= '0'; wait until (rising_edge(clk));      -- IN r1              -- r1 = 03
        IR <= X"4280"; wait until (rising_edge(clk));   -- IN r2              -- r2 = 05
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"02D1"; wait until (rising_edge(clk));   -- ADD r3, r2, r1      -- r3 = 08
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0AC2"; wait until (rising_edge(clk));   -- SHL r3, 2          -- r3 = 32
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"068B"; wait until (rising_edge(clk));   -- MUL r2, r1, r3      -- r2 = 96
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"4080"; wait until (rising_edge(clk));   -- OUT r2            -- r2 = 96
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        IR <= X"0000"; wait until (rising_edge(clk));   -- NOP
        rst <= '1';
        wait;
    end process;
```

The result:



`

There is a small delay or "lag" between the calculation of the inputs and intermediate output, (in1, in2 -> out1) because the output is being updated in the next clock cycle. To elaborate, the inputs (in1 and in2) are latched into the register (read by the ALU) on a clock edge, the ALU then updates the output on a subsequent clock edge (which is usually one or more clock cycles later). It is how synchronous logic work: everything is driven by the clock, so the next stage's outputs can't settle until the next clock cycle. A synchronous pipeline (or CPU) always has at least one clock cycle of delay between inputs changing and the final outputs being valid. We can observe that this will be a common behaviour for the rest of the test plans.

4.2 Format B

Similarly to Format A's test, the testbenches were first transformed into VHDL language.

Test 1

The purpose of this code is to test the processor's branching capabilities, including subroutine calls, conditional branches, and loop execution without data dependencies

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4  use work.all;
5
6
7  entity test_controller_sub is end test_controller_sub;
8
9  architecture behavioural of test_controller_sub is
10     component CONTROLLER_file port(
11         --input signals
12         rst: in std_logic;
13         clk: in std_logic;
14         IR: in std_logic_vector(15 downto 0);
15         --output signal
16         z, n, o: out std_logic;
17         PC: out std_logic_vector(15 downto 0);
18         out_val: out std_logic_vector(15 downto 0)
19     );
20     end component;
21     signal rst, clk, z, n, o: std_logic;
22     signal IR, PC, out_val : std_logic_vector(15 downto 0);
23
24     begin
25     u0 : CONTROLLER_file port map(rst, clk, IR, z, n, o, PC, out_val);
26
27     process begin
28         clk <= '0'; wait for 10 us;
29         clk <= '1'; wait for 10 us;
30     end process;
31
32     process begin
33         rst <= '1'; IR <= X"4200"; wait until (falling_edge(clk));
34         rst <= '0'; wait until (falling_edge(clk));     -- IN R0                 -- R0 = 02
35         IR <= X"4240"; wait until (falling_edge(clk));  -- IN R1                 -- R1 = 03
36         IR <= X"4280"; wait until (falling_edge(clk));  -- IN R2                 -- R2 = 01
37         IR <= X"42C0"; wait until (falling_edge(clk));  -- IN R3                 -- R3 = 05
38         IR <= X"024A"; wait until (falling_edge(clk));  -- ADD R1, R1, R2        -- R1 = 04
39         IR <= X"0488"; wait until (falling_edge(clk));  -- SUB R2, R1, R0        -- R2 = 02
40         IR <= X"045A"; wait until (falling_edge(clk));  -- SUB R1, R3, R2        -- R1 = 03
41         IR <= X"4040"; wait until (falling_edge(clk));  -- OUT r1                -- R1 = 03
42         IR <= X"4080"; wait until (falling_edge(clk));  -- OUT r2                -- R2 = 02
43         wait;
44     end process;
45 end behavioural; -- Test bench Format B part 1
```

Figure 29: Format A - test bench 1

`

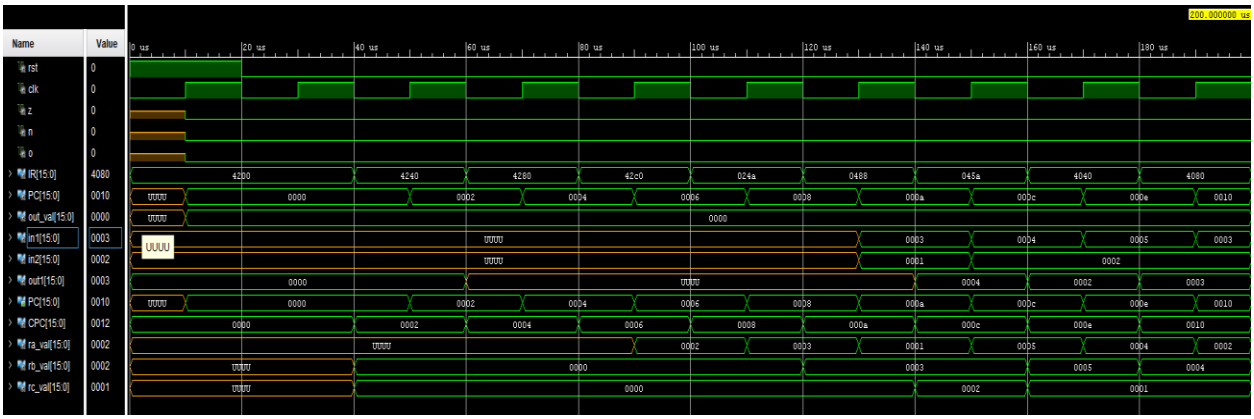


Figure 30: Format A - Test Bench 1 Result

Part 2

The purpose of this code is to test the processor's ability to handle subroutine calls, absolute and relative branching, and loop control using a counter with conditional branching based on the zero flag.

```
entity test_controller_sub is end test_controller_sub;
architecture behavioural of test_controller_sub is
    component CONTROLLER_file port(
        --input signals
        rst: in std_logic;
        clk: in std_logic;
        IR: in std_logic_vector(15 downto 0);
        --output signal
        z, n, o: out std_logic;
        PC: out std_logic_vector(15 downto 0);
        out_val: out std_logic_vector(15 downto 0)
    );
    end component;

    signal rst, clk, z, n, o : std_logic;
    signal IR, PC, out_val : std_logic_vector(15 downto 0);

    begin
    u0 : CONTROLLER_file port map(rst, clk, IR, z, n , o, PC, out_val);

    process begin
        clk <= '0'; wait for 10 us;
        clk <= '1'; wait for 10 us;
    end process;

    process begin
        rst <= '1'; IR <= X"4200"; wait until (falling_edge(clk));
        rst <= '0'; wait until (falling_edge(clk));     -- IN R0                                  -- R0 = 02
        IR <= X"4240"; wait until (falling_edge(clk));  -- IN R1                                  -- R1 = 03
        IR <= X"4280"; wait until (falling_edge(clk));  -- IN R2                                  -- R2 = 01
        IR <= X"42C0"; wait until (falling_edge(clk));  -- IN R3                                  -- R3 = 05
        IR <= X"4300"; wait until (falling_edge(clk));  -- IN R4                                  -- R4 = 528
        IR <= X"4340"; wait until (falling_edge(clk));  -- IN R5                                  -- R5 = 01
        IR <= X"4380"; wait until (falling_edge(clk));  -- IN R6                                  -- R6 = 05
        IR <= X"43C0"; wait until (falling_edge(clk));  -- IN R7                                  -- R7 = 00
        IR <= X"8D0A"; wait until (falling_edge(clk));  -- BR.SUB R4, 10                          -- R7 <- PC + 2 ; PC <- R4 + 2*(10)
        IR <= X"8000"; wait until (falling_edge(clk));  -- BRR 0                                  -- PC <- PC' + 2*(0) a.k.a infinite loop
        IR <= X"028D"; wait until (falling_edge(clk));  -- ADD R2, R1, R5                         -- R2 <- R1 + 1 = 04
        IR <= X"0642"; wait until (falling_edge(clk));  -- MUL R1, R0, R2                         -- R1 = R0 * R2 = 08
        IR <= X"05B5"; wait until (falling_edge(clk));  -- SUB R6, R6, R5                         -- R6 <- R6 - 1 = 04
        IR <= X"0F80"; wait until (falling_edge(clk));  -- TEST R6                                -- set z flag for branch
        IR <= X"8402"; wait until (falling_edge(clk));  -- BRR.z 2                                -- branch if r6 = 0
        IR <= X"81FB"; wait until (falling_edge(clk));  -- BRR -5                                 -- PC <- PC + 2*(-5)
        IR <= X"8E00"; wait until (falling_edge(clk));  -- RETURN                                 -- PC <- r7 a.k.a exit the subroutine
        IR <= X"4040"; wait until (falling_edge(clk));  -- OUT r1                                 -- R1 = 08
        IR <= X"4080"; wait until (falling_edge(clk));  -- OUT r2                                 -- R2 = 04
        IR <= X"4180"; wait until (falling_edge(clk));  -- OUT r6                                 -- R6 = 04
        IR <= X"41C0"; wait until (falling_edge(clk));  -- OUT r7                                 -- R7 = 16
        wait;
    end process;
end behavioural; -- TEST BENCH Format B Part 2
```

Figure 31: Format A - test bench 2

`

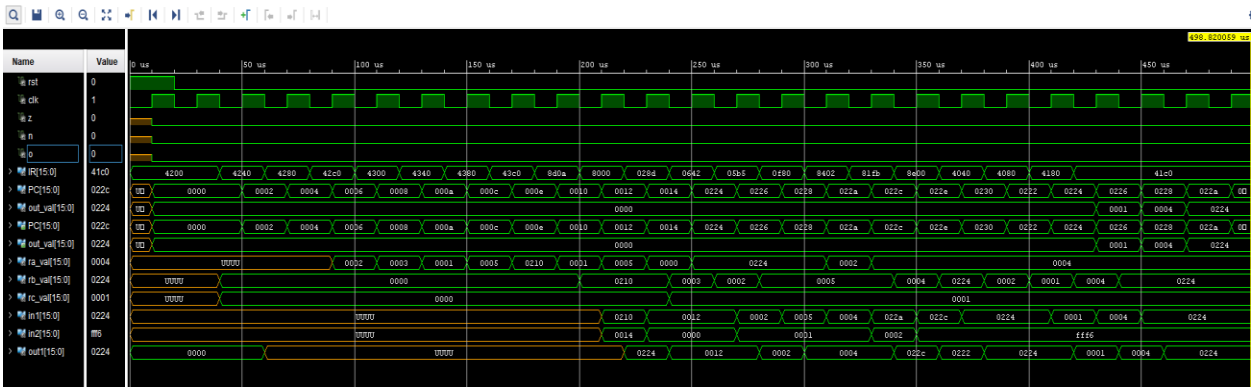


Figure 32: Format A - Test Bench 2 Result

Part 3

The purpose of this code is to test the execution speed and handling of a multiplication operation by the processor.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use work.all;

entity test_controller_sub is end test_controller_sub;

architecture behavioural of test_controller_sub is
    component CONTROLLER_file port(
        --input signals
        rst: in std_logic;
        clk: in std_logic;
        IR: in std_logic_vector(15 downto 0);
        --output signal
        z, n, o: out std_logic;
        PC: out std_logic_vector(15 downto 0);
        out_val: out std_logic_vector(15 downto 0)
    );
    end component;

    signal rst, clk, z, n, o : std_logic;
    signal IR, PC, out_val : std_logic_vector(15 downto 0);

    begin
    u0 : CONTROLLER_file port map(rst, clk, IR, z, n, o, PC, out_val);

    process begin
        clk <= '0'; wait for 10 us;
        clk <= '1'; wait for 10 us;
    end process;

    process begin
        rst <= '1'; IR <= X"4200"; wait until (rising_edge(clk));
        rst <= '0'; wait until (rising_edge(clk));     -- IN R0            -- R0 = -02
        IR <= X"4240"; wait until (rising_edge(clk));  -- IN R1            -- R1 = 03
        IR <= X"4280"; wait until (rising_edge(clk));  -- IN R2            -- R2 = 01
        IR <= X"42C0"; wait until (rising_edge(clk));  -- IN R3            -- R3 = 05
        IR <= X"0783"; wait until (rising_edge(clk));  -- MUL R6, R0, R3    -- R6 = R0 * R3 = -2 * 5 = - 10
        wait until (rising_edge(clk));                 -- wait 1 cycle for IN to read from memory
        IR <= X"4180"; wait until (rising_edge(clk));  -- OUT r6
        IR <= X"8000"; wait until (falling_edge(clk));  -- BRR 0            -- PC <- PC' + 2*(0) a.k.a infinite loop

        wait;
    end process;
end behavioural; -- Test bench Format B part 3
```

Figure 33: Format A - Test bench 3

`

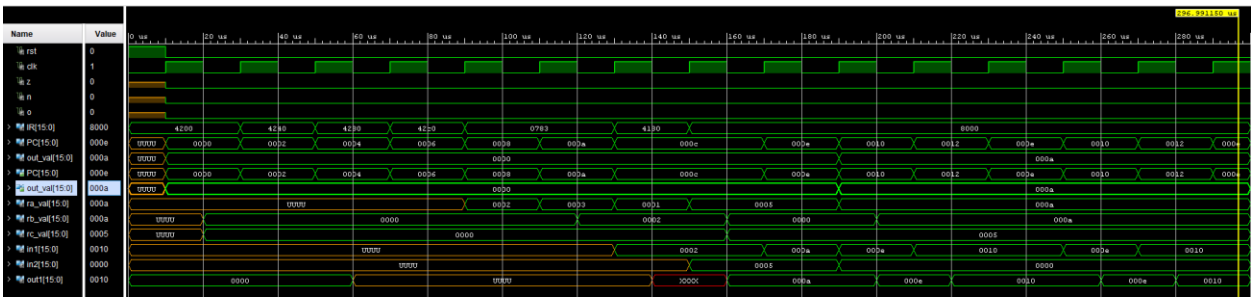


Figure 34: Format A - Test Bench 3 Result

4.3 Format L and Final Demo

Format L would be the combination of all of the mentioned format, which would ultimately the final test that the project required. As this stage, the CPU was not simply expected to run behaviour simulation. Rather, it was expected to successfully run Synthesize and Implementation in Vivaldo to verify all of the connections and gates worked as planned. To run the program, the following steps are required:

Run Synthesize -> Run Implementation -> Generate the Bitstream -> Program the device.

There are two main ways to run the program, the first one requires the memory file to be change directly in the program, the second one requires using a Boot loader. Before running the test, the lab manual provides a Python program to convert Assembly file into memory file and instruction file for the ROM of the program and the Bootloader. To do this, simply use the following line in the Git Bash terminal of the Python program: ./assembler.py -o <output file name> <input assembly code file name>. The Python program will output three files (.coe, .hex, .lst), with .lst being the only file needed.

Note*: Bootloader is discussed in detail later in the report.

Accessing the Test directly from ROM

After receiving three files from the Python program, a memory type file (.mem) has to be created. To this, from the .lst file received, in this case factorial.asm was being extracted, retrieve the following information being highlighted by the arrow. The first column would be the CPC counter, the second column would be the data or instruction.

`

```
DEPTH 1024
WIDTH 16
DEFAULT 0
;
; Data Section
; Specifies data to be stored in different addresses
; e.g., DATA 0:A, 1:0
;
RADIX 16
DATA
                          , -- .DATA
                          , -- .CODE
0000 => "0010010111111111", -- 0000 - 25FF main:          loadimm.upper DipSwitches.hi
0002 => "0010010011110000", -- 0002 - 24F0                loadimm.lower DipSwitches.lo
0004 => "0010000110111000", -- 0004 - 21B8                load          r6,r7
0006 => "0010010100000000", -- 0006 - 2500                loadimm.upper DipSwitchMask.hi
0008 => "0010010000001111", -- 0008 - 240F                loadimm.lower DipSwitchMask.lo
0010 => "0000100110110111", -- 000A - 09B7                nand          r6,r6,r7
0012 => "0000100110110110", -- 000C - 09B6                nand          r6,r6,r6
0014 => "0010010100000000", -- 000E - 2500                loadimm.upper 0x00
0016 => "0010010000000001", -- 0010 - 2401                loadimm.lower 0x01
0018 => "0010011100111000", -- 0012 - 2738                mov           r4,r7
0020 => "0010011011111000", -- 0014 - 26F8                mov           r3,r7
0022 => "0000111110000000", -- 0016 - 0F80                test          r6
0024 => "1000010000001101", -- 0018 - 840D                brr.z         Done
0026 => "0000010110110011", -- 001A - 05B3                sub           r6,r6,r3
0028 => "0000111110000000", -- 001C - 0F80                test          r6
0030 => "1000010000001010", -- 001E - 840A                brr.z         Done
0032 => "0010010100000000", -- 0020 - 2500                loadimm.upper 0x00
0034 => "0010010000000010", -- 0022 - 2402                loadimm.lower 0x02
0036 => "0010011101111000", -- 0024 - 2778                mov           r5,r7
0038 => "0000011100100101", -- 0026 - 0725 loop:          mul           r4,r4,r5
0040 => "0000001101101011", -- 0028 - 036B                add           r5,r5,r3
0042 => "0000010110110011", -- 002A - 05B3                sub           r6,r6,r3
0044 => "0000111110000000", -- 002C - 0F80                test          r6
0046 => "1000010000000010", -- 002E - 8402                brr.z         Done
0048 => "1000000111111011", -- 0030 - 81FB                brr           loop
0050 => "0010010111111111", -- 0032 - 25FF Done:          loadimm.upper LedDisplay.hi
0052 => "0010010011110010", -- 0034 - 24F2                loadimm.lower LedDisplay.lo
0054 => "0010001111100000", -- 0036 - 23E0                store         r7,r4
0056 => "1000000111111101", -- 0038 - 81FD                brr           Done


-------------------------------------------------------
Symbol Table:
CODE               0 (0000)
DipSwitchMask     15 (000F)
DipSwitches    65520 (FFF0)
Done              50 (0032)
LedDisplay     65522 (FFF2)
loop              38 (0026)
main               0 (0000)
```

Figure 35: .lst file showing CPC and instruction data

From here, create the following .mem file:

```
@0000 25FF
@0002 24F0
@0004 21B8
@0006 2500
@0008 240F
@000A 09B7
@000C 09B6
@000E 2500
@0010 2401
@0012 2738
@0014 26F8
@0016 0F80
@0018 840D
@001A 05B3
@001C 0F80
@001E 840A
@0020 2500
@0022 2402
@0024 2778
@0026 0725
@0028 036B
@002A 05B3
@002C 0F80
@002E 8402
@0030 81FB
@0032 25FF
@0034 24F2
@0036 23E0
@0038 81FD
```

Figure 36: Generated .mem file content example

`

After that load the memory file into the program by changing the input for ROM

Project Summary | console.vhd | ROM.vhd * | constraints.xdc | TopModule.vhd | RAM.vhd | factorial.mem

\\samba4.engr.uvic.ca\hoangtuankietvu\Desktop\449\lamlaiemoi\CPU_design\testLFormat\Everything\ROM.vhd

```
20  end ROM_file;
21
22  architecture behavioural of ROM_file is
23      begin
24
25      xpm_memory_sprom_inst : xpm_memory_sprom
26      generic map (
27
28          -- Common module generics
29          MEMORY_SIZE             => 8192,            -- 1024 bytes block
30          MEMORY_PRIMITIVE        => "auto",          --string; "auto", "distributed", or "block";
31          MEMORY_INIT_FILE        => "test1.mem",          --string; "none" or "<filename>.mem"
32          MEMORY_INIT_PARAM       => "",              --string;
33          USE_MEM_INIT            => 1,               --integer; 0,1
34          WAKEUP_TIME             => "disable_sleep", --string; "disable_sleep" or "use_sleep_pin"
35          MESSAGE_CONTROL         => 0,               --integer; 0,1
36          ECC_MODE                => "no_ecc",        --string; "no_ecc", "encode_only", "decode_only" or "both_
37          AUTO_SLEEP_TIME         => 0,               --Do not Change
38          MEMORY_OPTIMIZATION     => "true",          --string; "true", "false"
39
40          -- Port A module generics
41          READ_DATA_WIDTH_A       => 16,              --positive integer
42          ADDR_WIDTH_A            => 16,               --positive integer
43          READ_RESET_VALUE_A      => "0",             --string
44          READ_LATENCY_A          => 1                --non-negative integer
45      )
46      port map (
```

Figure 37: ROM configuration set to load memory file

Using the Bootloader

There more involved to this than just loading the ROM with a .mem file. There is a more dynamic and modular method to tun programs on the custom process that requires using a boot loader. The boot loader allows the programs to be remotely transferred to the FPG's RAM memory through the STM320F0 board without needing to recompile or rebuild the core each time. This process is made up of following five steps:

1. Assembling the Test Program: The first step involves converting the .asm assembly file into a .lst and .hex format using the command-line tool assembler19.py. This tool is executed using the command:

   python assembler19.py -o factorial factorial.asm

   The .lst file is used by the boot loader Python script, while the .hex file is intended for ROM import if needed. It is essential to omit the file extension when naming the output file [1].

2. Transferring the Program to the STM32F0 Board: Once the .lst file is generated, it is transferred to the STM32F0 using the FPGA Loader program located in the ece449 folder on the desktop. The loader utilizes a packet-based approach, where packets contain either address or data instructions, ensuring efficient memory mapping [1].

`

3. Resetting the CPU into Load Mode (0x0002):With the STM32F0 connected to the FPGA, the processor is reset into load mode by pointing the program counter to address 0x0002. In this mode, incoming packets are written into the CPU's RAM starting from a predefined address. This allows the instruction stream to be stored dynamically during runtime [1].
4. Loading the Code into RAM: The loader transmits each instruction to memory. Address packets are inserted at the beginning and after each ORG directive, while data packets are used for individual instructions. This mechanism supports proper sequencing and initialization [1].
5. Resetting the CPU into Execute Mode (0x0000): After loading is complete, the CPU is reset again—this time into execute mode by pointing to address 0x0000. The boot loader automatically redirects execution to the correct instruction starting point, validated via boot vector instructions stored in the first three RAM words [1].

The code that is used for the boot loader occupies the first few instructions in RAM and then sets up the correct jump to the user's code It also manages the dual reset vector support which makes sure that a reset into load or execute mode is recognized. Using the boot loader offers several advantages:

- It avoids the need to manually reprogram the ROM for each test [1].
- It supports real-time testing and debugging [1].
- It facilitates smoother workflow when updating or testing multiple assembly programs [1].

The boot loader is an efficient and flexible solution for loading and executing custom assembly program on the FPGA board. It closes the distance between the development environment and the hardware execution which reduces the setup time and improving the overall performance along with usability. As mentioned previously, the bootloader requires the .lst file after the extraction and it's important to note that the STM board will not accept more than 10 KHz clock speed (see Figure 35)

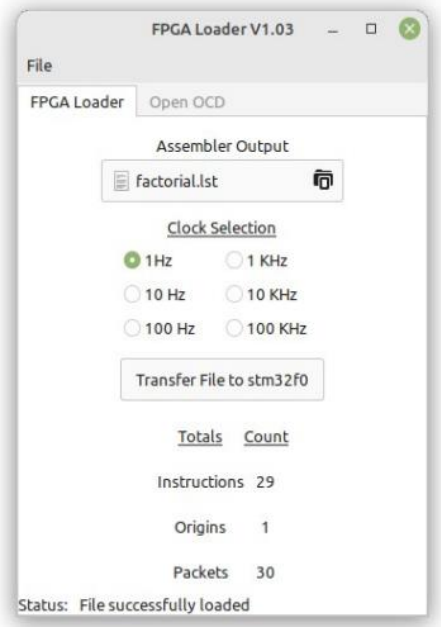


Figure 38: FPGA Loader

`

Final Result:

Both of the mentioned types would yield the same result, as part of the final test, VHDL code-based test benches was not created, we simply use the mentioned approach to perform the final test.

Before running the final test, a simple test was created to check the successful functionality of the console, 7 display segment, and more importantly the program.

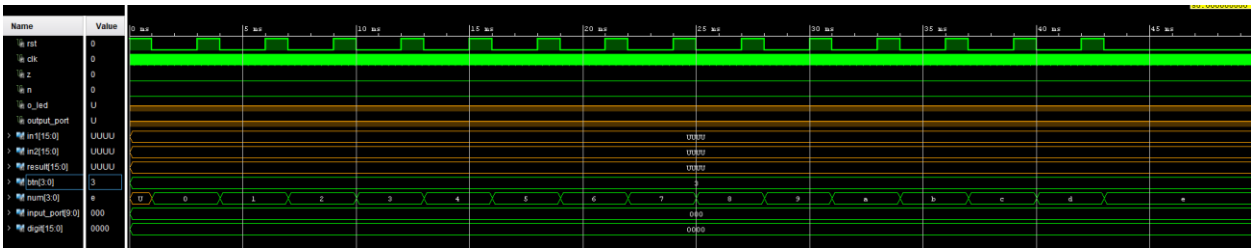


Figure 39: Vivado showing successful implementation summary

For the sake of keeping this report not to lengthy, only the factorial.asm file was used to carry out the test. Please note that other tests were already conducted on the final demo day.

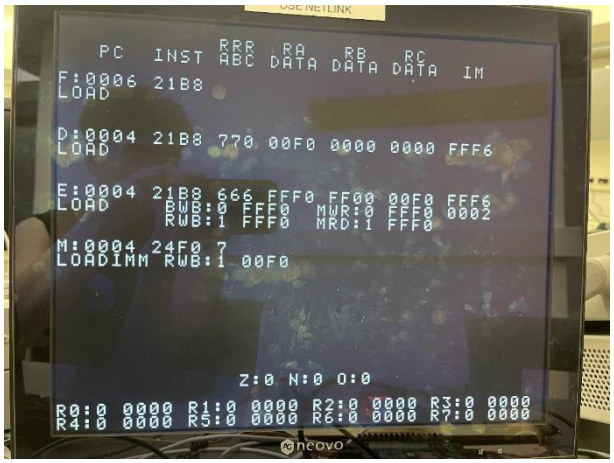


Figure 40: Bitstream generation completed in Vivado - 1

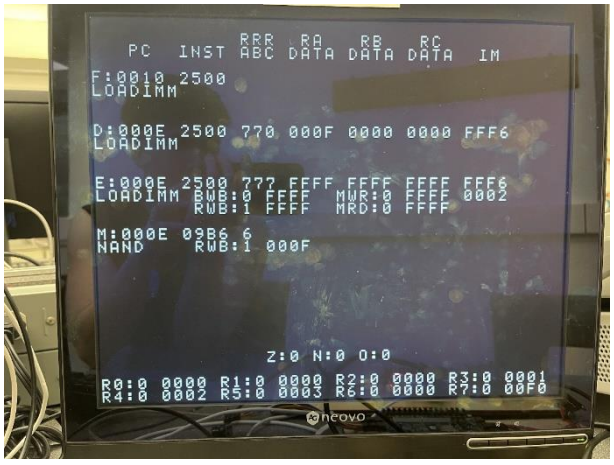


Figure 41: Bitstream generation completed in Vivado - 2

`

It can be observed that the displayed result aligned with the expected results from the asm file. Which conclude that his project was a success.

5.0 Discussion:

The pipelined processor architecture was successfully implemented and verified across Format A, Format B, and Format L. This was verified by using comprehensive waveforms simulation, VGA console outputs, and real time debugging interfaces. At the conclusion of each of these tests, it was confirmed that each pipeline stage and control signal operated as it was supposed to, and the results aligned with what was expected. The results are discussed in more detail below.

5.1 Format A:

Format A instructions are fundamental to the CPU's functionality. It validates the behaviour of both ALU and the general-purpose registers. Figure 35 and Figure 36 show the execution of an ADD operations, where the register values are read, processed by the ALU, and then written back. In Figure 35, inputs in1 and in2 are taken form their respective registers and out1 holds the result that was computed. The waveform confirms that the out1 becomes valid only after one clock cycle which is consistent with the processor's pipeline design. The clock cycle delay also aligns with what was expected, which confirms proper timing through the pipeline registers. During the Write Back stage, the we_en signal is asserted, updating ra_val with ALU result. Figure 36 also verifies that the content and instructions bits of the register are decoded accurately and forwarded to the ALU during execution. Moreover, the control flags such as o_flag, z_flag, and n_flag are correctly monitored and then updated during ALU operations. Figure 38 shows this, providing a look into the control logic's handling of edge case during the arithmetic computations.

5.2 Format B:

Moving on, Format B supports instructions such as LOAD, STORE, and MOV which validated the integration of RAM and memory mapped I/O withing the pipeline. The address computation phase is showing in Figure 37; here the ALU calculates the target memory address. A successful memory read (LOAD) is captured in Figure 39, with mem_dt driven onto the write back bus and then written to the destination register. During the STORE operations, the wr_mem_en control signal is used to enable memory write. This is confirmed in Figure 39. Where wr_mem_en activates in sync with din_dt, which verifies that the data is written to the correct address at the time it was supposed to. The MOV instruction, shown in Figure 40 shows a direct register to register data transfer. The ALU data is bypassed; however, the data still progresses through all pipeline stages. This confirms that the bypass logic worked properly and no unintended ALU operations took place. Also, Format B's support for the memory mapped I/O is verified through Figure 18 and Figure 19. These figures show the CPU interacting with the external peripherals. It specifically reads from the address 0xFFF0 for keypad input and writing to 0xFFF2 to toggle LEDs. The IN and OUT instructions behave as they were expected to, which confirmed the successful communication between the processor and I/0 devices.

5.3 Format L:

Finally, Format L introduces the most difficult part of the project, it incorporates the dynamic control flow, immediate value handling, subroutine execution, and hazard resolution. Figure 40 shows the successfully execution of a BR.Z instruction. Here the z_flag is evaluated and the stall_MUX is used to

`

switch the instruction fetch source from ROM to RAM. This allows the immediate redirection of the program counter (PC) to the appropriate branch address. The branch behaviour is further verified in Figure 31, where the PC is correctly updated in the following cycle, and the invalid instructions are flushed from the pipeline. The waveform clearly shows the re-alignment of the fetch, decode, and execute stage during the branch transition. The LOADIMM instruction was then used to test the immediate value expansion logic via the SIGNEXT_file module. In Figure 32, and 8-bit immediate is correctly sign extended and shifted to the form a 16-bit constant, confirming the proper destination of register. The subroutine control instructions like BR.SUB and RETURN were also validated. As shown in Figure 33, register R7 is used as a link register to store the return address. Control signals brch_en and stall_MUX are activated to temporarily halt fetching and redirect execution accurately. The simulation confirmed correct branching behavior with no data hazards or pipeline corruption during subroutine execution.

5.4 Conclusion:

Over all of the instruction formats, signal behavior, output values, and control flow were verified through waveform inspection, debut console outputs, and register tracing. Figures 13, 22, 32 and 40 show the visual confirmation of instruction moving through the pipeline, showing accurate updates to control the signals, flags, and outputs. Each of the test targeted a specific architectural future which were validating ALU operations, memory interactions, hazard handling, and branch execution. The seamless coordination among pipeline components form PC_file and MUX_file to REGISTER_file and RAM_file; this demonstrates that the architecture operates reliable across diverse workloads and instruction types.

6.0 Limitation and Possible Improvements:

While the implemented pipelined architecture successfully executes Format A, Format B, and Format L instructions, several inherent limitations present opportunities for refinement and future development.

6.1 Debug Console Overhead:

The VGA based debug console which is implemented in the console.vhd and showing in Figures 31-33 highlight the pipeline's internal state but it also introduces resources and timing overhead. This can severely impact performance on the resource constrained FPGAs. To fix this, Modularizing the debug system and enabling it selectively through compile time flags or runtime control ultimately mitigate the overhead.

6.2 Memory Access Handling:

The LOAD/STORE Format B instruction sometimes suffer from timing mismatches, especially when they are interacting with special memory mapped I/O locations like 0xFFF0. Figure 41-43 show the execution delays caused by RAM latency and clock alignment issues. Enhancing the memory interface with acknowledgment-based handshaking or multi cycle access support would improve timing accuracy and system responsiveness.

6.3 Overflow and Flag Register Handling:

Currently, the overflow and conditions flags are maintained using per-register signals (like o_flags array), which complicates the scaling and decoding. Introducing a centralized flag register, updated during ALU operations and visible to conditional instructions would streamline the flag management and simplify control logic for branching and comparison operations.

`

7.0 Summary:

This project posed a lot of challenges due to the lack of a step-by-step guide that is usually provided for many other labs. However, not having a step-by-step guide gave the team a deeper understanding of the processor design which encouraged independent debugging and also reinforced system level integration skills. As a result, the team was successful in designing and implementing a pipeline processor that is capable of executing three distinct instruction formats (Format A, Format B, and Format L). All of the functions that were required for this project were meant and the expected results were achieved according to the project specifications and were previously highlighted in Section 4 and Section 5. It's also important to note that only a single reference link was used throughout the project because all of the necessary resources and documentation were provided on the lab's designated website. The full code is available on GitHub, the link is available in the references. Finally, Team Asia would like to extend our sincere thanks to Dr. Amirali Baniasadi for his guidance, Lab Technician Brent Sirna for supplying key design tools and simulation support, and Teaching Assistant Aaron Chegini for his ongoing assistance and valuable insights throughout the semester. As Hoan, Phillip, and Huss complete their Bachelors Degree they would like to wish Aaron all the best as he pursue his Maste's Degree in Machine Learning.

`

8.0 References:

[1] University of Victoria, “ECE 449 Laboratory Resources,” *Department of Electrical and Computer Engineering*, [Online]. Available: https://www.ece.uvic.ca/~ece449/lab/index.html. [Accessed: Apr. 4, 2025].

9.0 Appendix:

https://github.com/Liuphillip/449-project